

Digital Logic Design & Verification

Half Adder | Full Adder | 2:1 Multiplexer — ModelSim Simulation Project

Author	Murali Krishna
Program	Cognifyz Internship
Tool Used	ModelSim (HDL Simulator)
Language	Verilog HDL
Modules Covered	Half Adder, Full Adder, 2:1 Multiplexer

This document summarizes the design, implementation, and functional verification of three fundamental combinational logic circuits — the Half Adder, Full Adder, and 2:1 Multiplexer — written in Verilog HDL and verified using ModelSim.

1. Introduction

This project was carried out as part of the Cognifyz internship to build a strong foundation in digital design and functional verification using an industry-standard HDL simulator. Three combinational logic circuits were designed in Verilog, each accompanied by a self-checking or stimulus-driven testbench, and verified in ModelSim by observing the generated waveform against the expected truth table.

The general workflow followed for each module was:

1. Write the RTL design module in Verilog.
2. Write a testbench to apply input stimulus.
3. Compile the design and testbench in ModelSim.
4. Simulate and observe the waveform in the Wave window.
5. Cross-check simulation output against the truth table.

2. Tools and Environment

Tool / Item	Details
Simulator	ModelSim (Intel/Altera Starter Edition or equivalent)
HDL	Verilog
Design Style	Combinational (dataflow / gate-level modeling)
Verification Method	Testbench with applied input vectors, waveform inspection

3. Half Adder

A Half Adder adds two single-bit binary numbers and produces a Sum and a Carry output. It does not account for a carry input from a previous stage.

3.1 Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

3.2 Boolean Equations

Sum = A XOR B Carry = A AND B

3.3 Verilog Design Module

```
module half_adder(  
    input A,  
    input B,  
    output Sum,
```

```

        output Carry
    );
    assign Sum = A ^ B;
    assign Carry = A & B;
endmodule

```

3.4 Testbench

```

module half_adder_tb;
    reg A, B;
    wire Sum, Carry;

    half_adder uut (.A(A), .B(B), .Sum(Sum), .Carry(Carry));

    initial begin
        $display("A B | Sum Carry");
        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;
        $stop;
    end

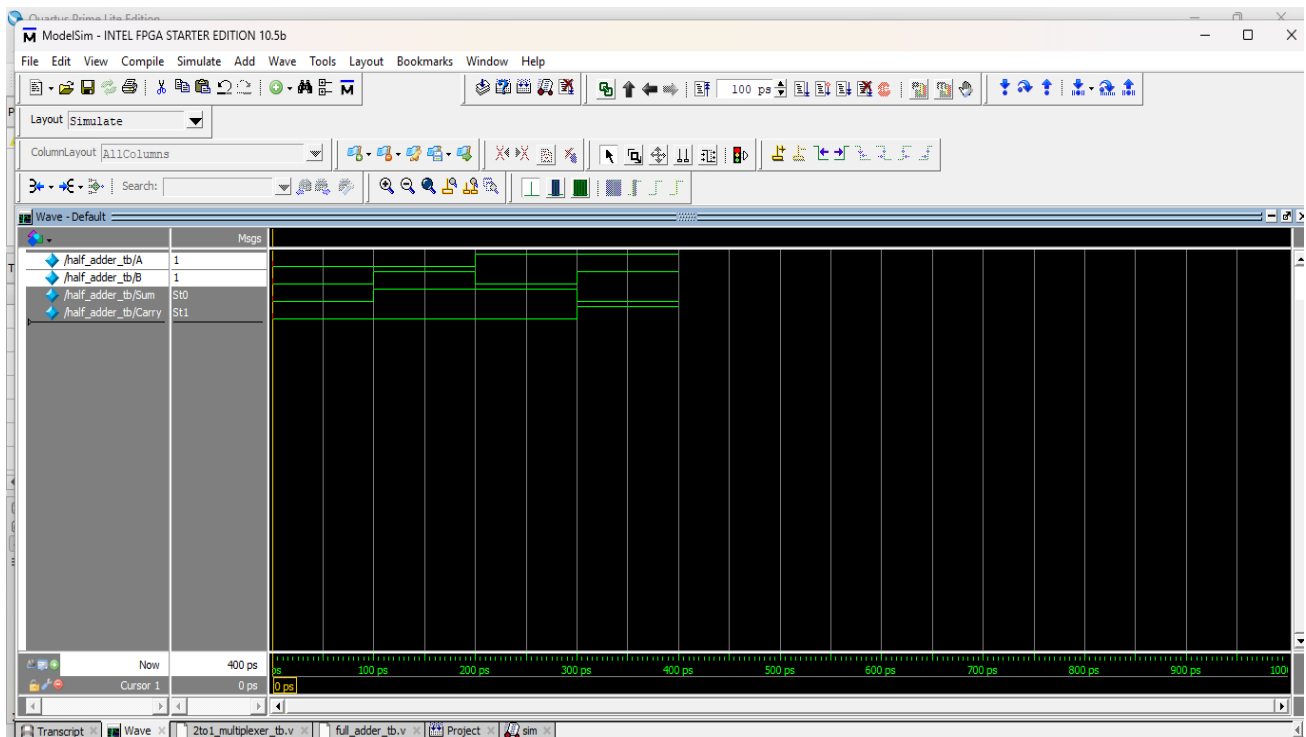
    initial
        $monitor("%b %b | %b %b", A, B, Sum, Carry);
endmodule

```

3.5 Simulation Result

The waveform observed in ModelSim's Wave window matched the truth table exactly: Sum toggled high whenever A and B differed, and Carry went high only when both A and B were 1. *[Insert your ModelSim waveform screenshot here before uploading to GitHub.]*

Half Adder Output wave forms



4. Full Adder

A Full Adder extends the Half Adder by including a Carry-in (Cin) input, allowing multiple adders to be cascaded to build multi-bit adders.

4.1 Truth Table

A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4.2 Boolean Equations

Sum = A XOR B XOR Cin Carry = (A AND B) OR (A AND Cin) OR (B AND Cin)

4.3 Verilog Design Module

```
module full_adder(
    input A,
    input B,
    input Cin,
    output Sum,
    output Carry
);
    assign Sum = A ^ B ^ Cin;
    assign Carry = (A & B) | (A & Cin) | (B & Cin);
endmodule
```

4.4 Testbench

```
module full_adder_tb;
    reg A, B, Cin;
    wire Sum, Carry;

    full_adder uut (.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Carry(Carry));

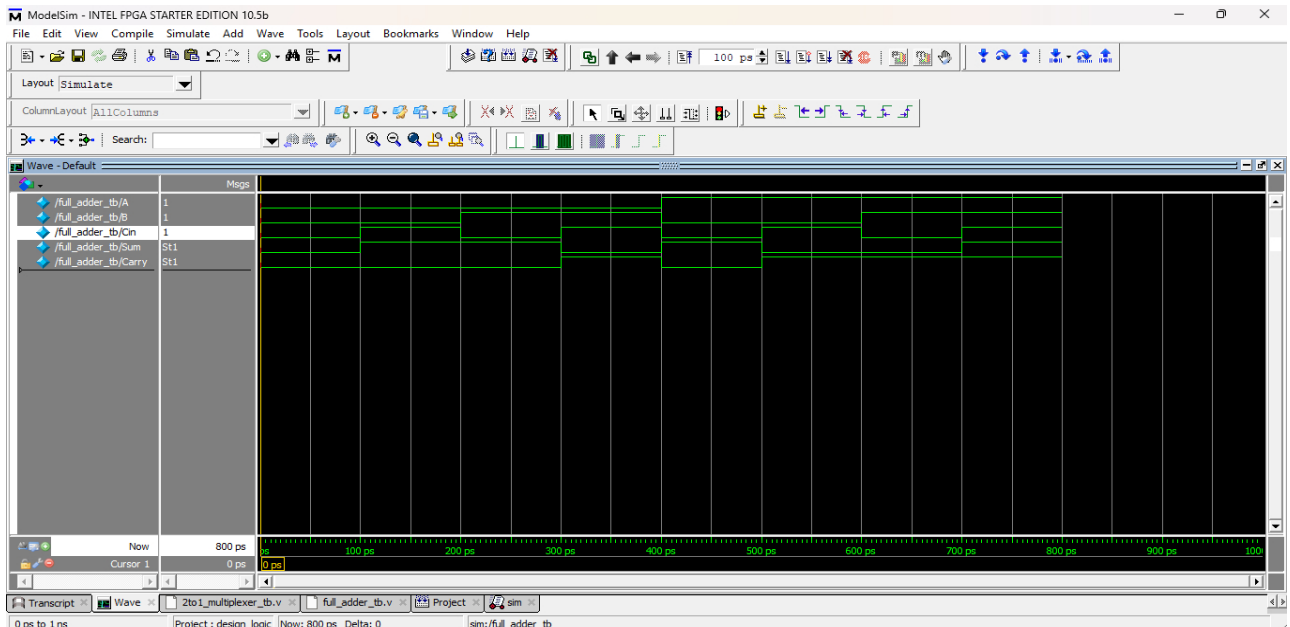
    initial begin
        A = 0; B = 0; Cin = 0;
        repeat (8) begin
            #10 {A, B, Cin} = {A, B, Cin} + 1;
        end
        #10 $stop;
    end

    initial
        $monitor("A=%b B=%b Cin=%b | Sum=%b Carry=%b", A, B, Cin, Sum, Carry);
endmodule
```

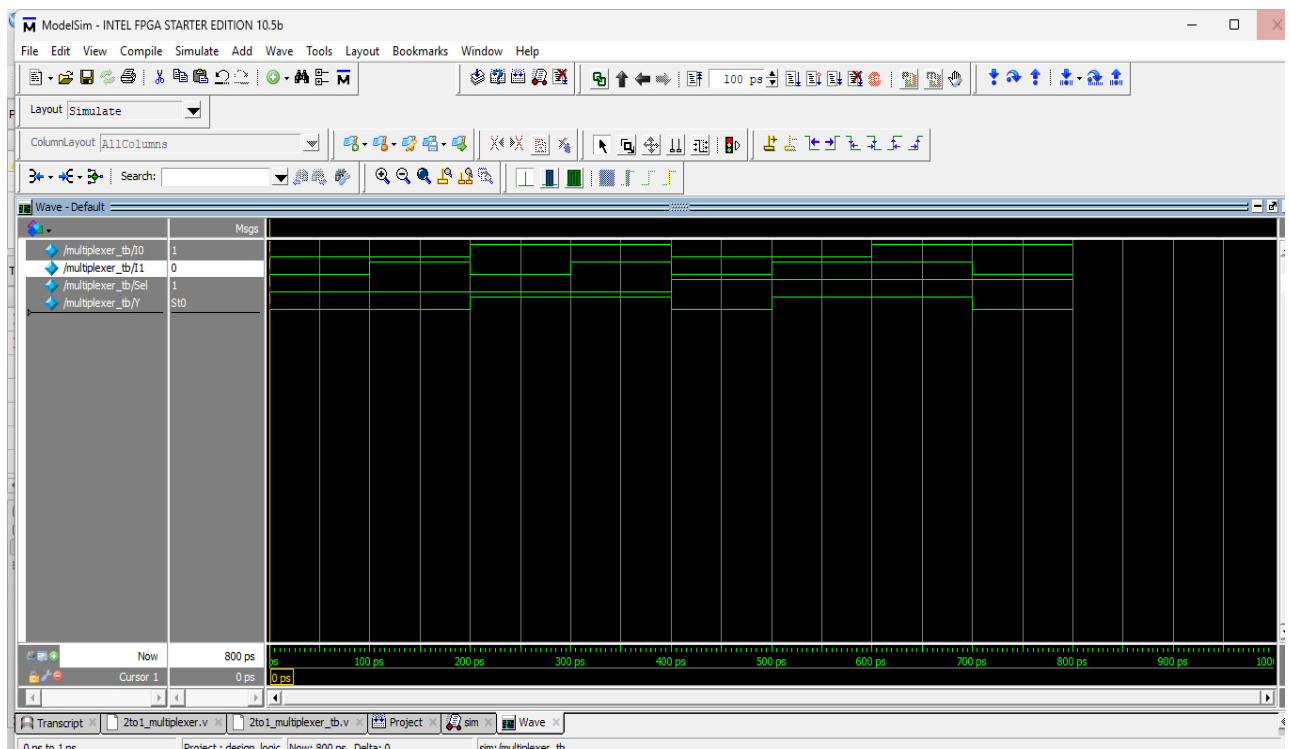
4.5 Simulation Result

All eight input combinations were applied using an incrementing counter on the concatenated {A, B, Cin} bus. The Sum and Cout waveforms matched the expected truth table for every combination. *[Insert your ModelSim waveform screenshot here before uploading to GitHub.]*

Full Adder Output Waveforms



Simulation Result for 2:1 Multiplexer



5. 2:1 Multiplexer

A 2:1 Multiplexer selects one of two input lines (I0, I1) and routes it to the output Y, based on the value of a single select line (Sel).

5.1 Truth Table

Sel	I0	I1	Y
0	x	-	I0
1	-	x	I1

(When Sel = 0, Y follows I0 regardless of I1; when Sel = 1, Y follows I1 regardless of I0.)

5.2 Boolean Equation

$Y = (Sel' \text{ AND } I0) \text{ OR } (Sel \text{ AND } I1)$ — implemented in Verilog using the conditional (ternary) operator:
 $Y = Sel ? I1 : I0$

5.3 Verilog Design Module

```
module mux2x1(  
    input I0,  
    input I1,  
    input Sel,  
    output Y  
);  
assign Y = Sel ? I1 : I0;  
endmodule
```

5.4 Testbench

```
module mux2to1_tb;  
    reg I0, I1, Sel;  
    wire Y;  
  
    mux2x1 uut (.I0(I0), .I1(I1), .Sel(Sel), .Y(Y));  
  
    initial begin  
        I0 = 0; I1 = 1; Sel = 0; #10;  
        Sel = 1; #10;  
        I0 = 1; I1 = 0; Sel = 0; #10;  
        Sel = 1; #10;  
        $stop;  
    end  
  
    initial  
        $monitor("Sel=%b I0=%b I1=%b | Y=%b", Sel, I0, I1, Y);  
endmodule
```

5.5 Simulation Result

The waveform confirmed correct switching behavior: Y tracked I0 whenever Sel was low and switched to track I1 whenever Sel was driven high, with no glitches observed at the transition points. *[Insert your ModelSim waveform screenshot here before uploading to GitHub.]*

6. Suggested GitHub Repository Structure

```
ModelSim-Combinational-Logic/  
|-- README.md  
|-- docs/  
|   |-- ModelSim_Project_Documentation.pdf  
|-- half_adder/  
|   |-- half_adder.v  
|   |-- half_adder_tb.v  
|   |-- waveform_half_adder.png  
|-- full_adder/  
|   |-- full_adder.v  
|   |-- full_adder_tb.v  
|   |-- waveform_full_adder.png  
|-- mux2x1/  
|   |-- mux2x1.v  
|   |-- mux2x1_tb.v  
|   |-- waveform_mux2x1.png
```

Keep each module and its testbench together in its own folder, and add the corresponding waveform screenshot from ModelSim next to it so reviewers can see simulation proof without re-running the tools.

7. Conclusion

All three combinational circuits — Half Adder, Full Adder, and 2:1 Multiplexer — were successfully designed in Verilog and functionally verified in ModelSim. The simulation waveforms for each module matched their respective truth tables across all input combinations, confirming correct RTL behavior before any further synthesis or FPGA-level implementation.